

Column Generation Algorithm: Optimizing Large-Scale LP & IP Problems

A term Presentation for Crew Scheduling Problem

Presented by: Md. Inzamamul Lohani
Institute of Information Technology (IIT),
University of Dhaka

Supervisor: Shah Mostafa Khaled, Ph.D.
Institute of Information Technology (IIT),
University of Dhaka

Technologies Used

Languages or Tools to be used:

- Simplex Method
- Dual Problem

Programming Language: JAVA

Libraries: IBM CPLEX Optimization Studio

Tools: Git, VS Code

Platform: macOS

LOC: 750+

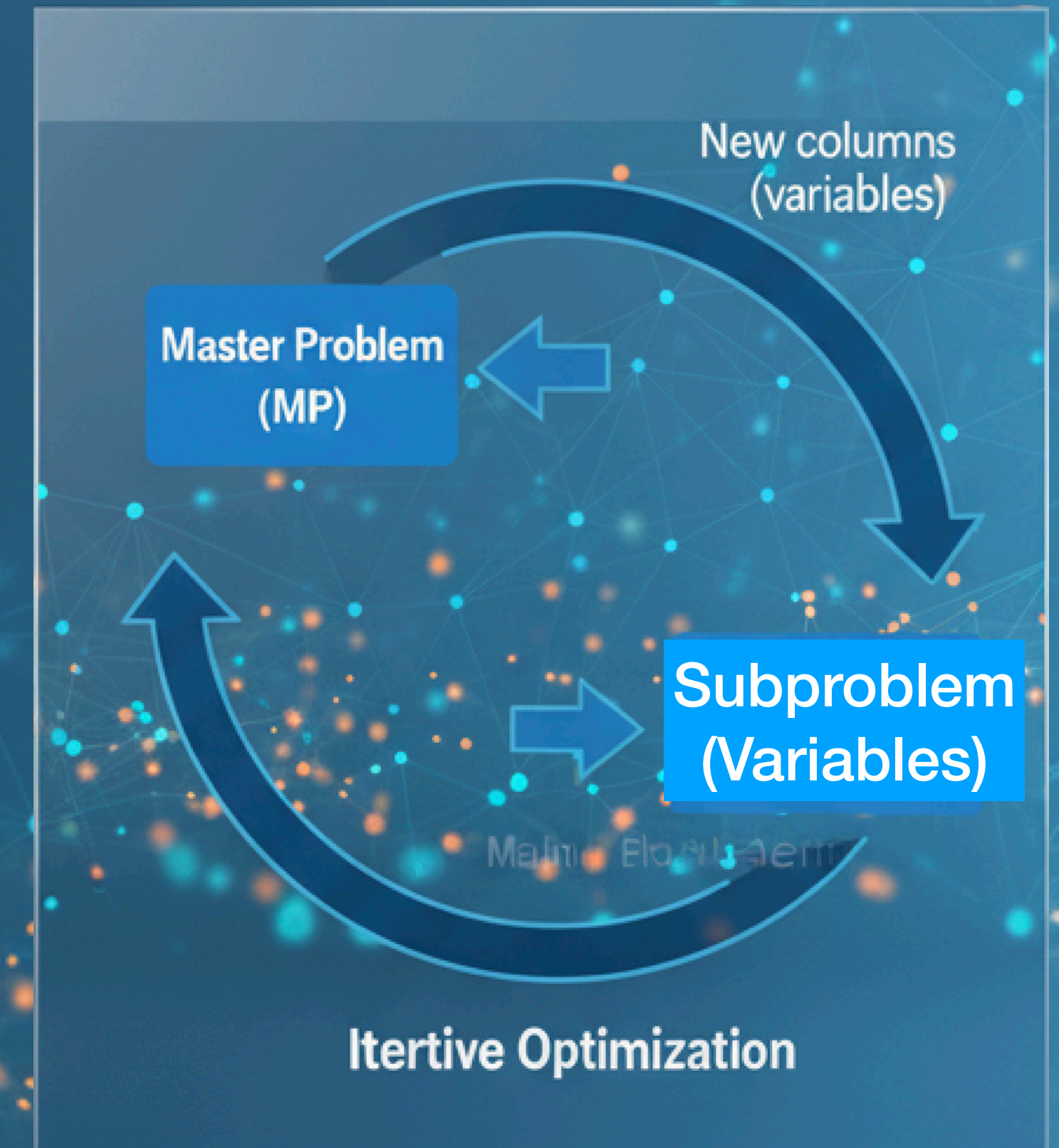
Finished: About 70%



Project Overview: Column Generation Algorithm For Large-Scale LP & IP Problems

CGA Decomposition:

- Decomposition technique for large-scale LPs
- It generates variables **only when needed**
- Instead of solving the full problem:
 - **Master Problem:** Start with a small subset of variables
 - **Sub Problem:** Iteratively add promising variables



Project Overview: Column Generation Algorithm For Large-Scale LP & IP Problems

- In optimal solutions:

- Most variables are zero (solution sparsity)
- Only a few variables affect optimality

- Column Generation:

- Identifies these useful variables
- Avoids generating useless ones

Column Generation Algorithm Steps

1. Initialize **RMP** with feasible columns
2. Solve **RMP**
3. Obtain **Dual Prices**
4. Solve **Pricing Problem(subproblem)**
5. Add negative **Reduced Cost** columns
6. Repeat until no improving column exists

Crew Scheduling Problem (Overview)

Objective:

- Assign crews to cover all flights at minimum cost
- Each crew follows labor and operational rules
- Each possible crew schedule is called a **pairing**

Why Crew Scheduling Is Hard

- A single pairing = one variable
- Number of feasible pairings is enormous

Constraints:

- Every flight must cover
- Start from base
- Duty hour limits
- Flight time limits
- Turn around

Standard Linear Programming Form

$$\min \sum_i C_i x_i$$

Subject to:

$$\sum_{p \in C} a_{ip} x_p \geq b_i$$

- i : very large variable set
- Explicit enumeration is impossible

```
1  for (Flight f : flights) {  
2      IloLinearNumExpr expr = cplex.linearNumExpr();  
3      IloRange constraint = cplex.addEq(expr, 1.0, "Cover_" + f.getFlightId());  
4      constraints.put(f, constraint);  
5  }
```

Adding constraints

Mathematical Model (Master Problem)

- Decision variable

$$x_i = \begin{cases} 1, & \text{flight done} \\ 0, & \text{not done} \end{cases}$$

- Objective

$$\min \sum_i C_i x_i$$

- Constraints

$$\sum_{p \in C} a_{ip} x_p \geq 1$$

Step:1 Restricted Master Problem (RMP)

- Start with a **small subset**
- Solve:

$$\min \sum_i C_i x_i$$

Subject to original constraints

- This is called the **Restricted Master Problem**

Restricted Master Problem in Crew Scheduling

- Start with:
 - Simple pairings
 - Single-flight or short pairings
- Solve LP
- Obtain dual prices for flights

```
1 public List<Pairing> getSolution() throws IloException {  
2  
3     List<Pairing> selected = new ArrayList<>();  
4  
5     for (Pairing p : columns) {  
6         double val = cplex.getValue(variables.get(p));  
7         if (val > 0.0001) {  
8             selected.add(p);  
9         }  
10    }  
11    return selected;  
12 }
```


Dual Variables (Shadow Prices)

Solving RMP gives:

- Primal variables
- Dual variables π

Dual variables represent:

- Marginal value of satisfying each constraint

```
1 public double[] getDUALS() throws IloException {  
2     double[] duals = new double[flights.size()];  
3     for (int i = 0; i < flights.size(); i++) {  
4         duals[i] = cplex.getDUAL(constraints.get(flights.get(i)));  
5     }  
6     return duals;  
7 }  
8  
9 }
```


Reduced Cost

For any variable i :

$$\text{Reduced cost} = C_i - \sum \pi_i$$

If reduced cost < 0 :

- Variable can improve solution

If all reduced costs ≥ 0 :

- Current solution is optimal (LP)

```
1 private double calculateReducedCost(Pairing p, java.util.Map<String, Double> duals) {  
2     double dualSum = 0;  
3     for (Flight f : p.getFlights()) {  
4         dualSum += duals.getOrDefault(f.getFlightId(), 0.0); //lib to get dualsum  
5     }  
6     return p.getCost() - dualSum;  
7 }  
8  
9  
10
```


Step:2 Pricing Problem(Sub Problem)

- Goal: find variables with **negative reduced cost**
- This is called the **Pricing Problem**
- It is problem-specific
- For scheduling problems:
 - Often a shortest path problem

Pricing Problem in Crew Scheduling

- Generate new crew pairings

- Objective:

$$\min \left(c_p - \sum_f \pi_f a_{fp} \right)$$

- This becomes:

- Shortest path problem with constraints

Handle the Constraints in Pricing Problem

Base, duty time, negative reduced cost

```

1 // check if we can close the pairing to Base
2     if (current.getTo().equals(base)) {
3         // check full duty validity & Cost
4         double dutyTime = calculateDutyTime(currentPath);
5
6         if (dutyTime <= maxDutyHours) {
7             Pairing p = createPairing(currentPath);
8             double redCost = calculateReducedCost(p, duals);
9             // negative reduced cost
10            if (redCost < -0.0001) {
11                solutions.add(p);
12            }
13        }
14    }

```

F1.To() == F2.From(), turnaround time

```

1 if (!f1.getTo().equals(f2.getFrom())) {
2     return false;
3 }
4
5 // time connection
6 long turn = TimeUtils.minutesBetween(f1.getArrTime(), f2.getDepTime());
7 if (turn < minTurnaroundMin) {
8     return false;
9 }

```


Column Generation Loop (Crew Scheduling)

1. Solve RMP
2. Get flight dual prices
3. Solve shortest path problem
4. Generate new pairing
5. Add to RMP
6. Repeat

Termination

- When no pairing has negative reduced cost
- RMP solution is LP-optimal

Final Output

Step 4: Column Generation Execution

```

Iter 1: Obj = 31000000.00 | Cols Added = 50 | Best RedCost = -8996600.00
Iter 2: Obj = 22003400.00 | Cols Added = 50 | Best RedCost = -7997180.00
Iter 3: Obj = 22003400.00 | Cols Added = 50 | Best RedCost = -5997448.00
Iter 4: Obj = 13364.00 | Cols Added = 50 | Best RedCost = -300.00
Iter 5: Obj = 13139.00 | Cols Added = 0 | Best RedCost = 0.00
  
```

STEP 5: FINAL OUTPUT

```

Total Cost: 13139.0
Execution Time: 118 ms
Iterations: 5
  
```

Selected Pairings:

```

F12->F13->F22->F23->F26->F27->F07->F08 ($2986.0)
F05->F06->F09->F10->F11 ($1960.0)
F18->F19->F22->F23 ($1556.0)
F07->F08->F18->F19 ($1612.0)
F05->F06->F16->F17 ($1630.0)
F01->F02->F14->F15 ($1726.0)
F12->F13->F16->F17 ($1740.0)
F03->F04->F09->F10->F11 ($2290.0)
F20->F21->F24->F25 ($1766.0)
F30->F31->F07->F08 ($1836.0)
F28->F29->F01->F02 ($1876.0)
F03->F04->F14->F15 ($1880.0)
F28->F29->F12->F13->F26->F27->F30->F31 ($3350.0)
F28->F29->F07->F08->F26->F27->F30->F31 ($3296.0)
  
```


Thank you